

Enterprise RAG Platform: Technical Requirements Document (TRD)

Table of Contents

1. Executive Summary
2. Business Goals
3. Technical Goals
4. System Scope
5. Assumptions
6. Constraints
7. Technology Selection
8. High-Level Architecture
9. Deployment Architecture
10. Component Architecture
11. Clean Architecture
12. Frontend Architecture
13. Backend Architecture
14. Database Design
15. Vector Database Design
16. Object Storage Design
17. Document Processing Pipeline
18. Chunking Strategy
19. Retrieval Pipeline
20. Conversation Management
21. Authentication
22. Authorization
23. API Design
24. Security Architecture
25. Performance Optimization

26. Scalability
27. Background Processing
28. Deployment Strategy
29. Observability
30. CI/CD
31. Testing Strategy
32. Coding Standards
33. Project Structure
34. Development Roadmap
35. Risk Analysis
36. Future Enhancements
37. References

1. Executive Summary

This document outlines the technical requirements, architecture, and design decisions for an enterprise-grade Retrieval-Augmented Generation (RAG) platform. The platform is designed to provide scalable, secure, and maintainable AI-powered search, document management, and conversational interfaces for enterprise clients. By leveraging a cloud-native architecture, the system ensures high availability, horizontal scalability, and robust security controls, adhering to standards seen in leading technology organizations.

2. Business Goals

The primary business objectives of the RAG platform are to:

- Enhance enterprise knowledge accessibility by providing semantic search capabilities.
- Reduce support and information retrieval times through AI-powered chat interfaces.
- Ensure data integrity and security through grounded responses with verifiable source citations.
- Support multi-tenancy to serve multiple organizations securely within a single platform.
- Provide a highly available and scalable infrastructure to support thousands of organizations and millions of users.

3. Technical Goals

The technical objectives focus on building a robust and extensible system:

- **Scalability:** Implement horizontal scaling for both frontend and backend services to handle high concurrent loads.
- **Security:** Enforce strict Role-Based Access Control (RBAC), secure document uploads, and protect against prompt injection and data leakage.
- **Maintainability:** Utilize Clean Architecture principles and SOLID design patterns to ensure modularity and ease of future enhancements.
- **Performance:** Optimize retrieval pipelines with efficient vector database queries, caching mechanisms, and streaming responses.
- **Observability:** Integrate comprehensive logging, metrics, and tracing to monitor system health and AI response quality.

4. System Scope

The platform encompasses the following core functionalities:

- **Document Management:** Uploading, processing, versioning, and storing enterprise documents.
- **Semantic Search:** Performing vector-based similarity searches across document corpora.
- **AI Chat Interface:** Providing conversational interactions powered by Large Language Models (LLMs) with streaming capabilities.
- **Multi-tenancy:** Isolating data and configurations per organization or workspace.
- **Authentication and Authorization:** Managing user identities, sessions, and granular access permissions.
- **Background Processing:** Handling asynchronous document parsing, OCR, embedding generation, and indexing.

5. Assumptions

- The target audience consists of enterprise users requiring high security and compliance standards.
- The system will primarily process text-based documents (PDFs, Markdown, plain text), with OCR support for scanned images.
- External dependencies (LLM providers, vector databases) will maintain their Service Level Agreements (SLAs).

- Users will have modern web browsers supporting the latest JavaScript and HTML5 standards.
- ## 6. Constraints
- The technology stack is strictly defined (Next.js 15, FastAPI, Neon PostgreSQL, Qdrant, Backblaze B2, Groq).
 - The frontend must be deployed on Vercel, and the backend on Railway or Render.
 - The embedding model is fixed to `sentence-transformers/paraphrase-MiniLM-L3-v2` for the initial release.
 - Latency requirements dictate that the retrieval and generation pipeline must provide streaming responses.

7. Technology Selection

Technology	Selection	Rationale & Trade-offs
Frontend	Next.js 15, React 19, TypeScript, Tailwind CSS, shadcn/ui, TanStack Query, Zustand	Rationale: Next.js provides excellent server-side rendering (SSR) and SEO capabilities, while React 19 offers enhanced concurrency. TypeScript ensures type safety. Tailwind CSS and shadcn/ui facilitate rapid, consistent UI development. TanStack Query handles server state caching, and Zustand provides lightweight client state management. Trade-offs: Next.js introduces a learning curve for complex SSR patterns compared to pure client-side SPAs.
Backend	FastAPI, Python 3.13+	Rationale: FastAPI offers high performance, automatic OpenAPI documentation, and native asynchronous support, making it ideal for streaming LLM responses. Python 3.13+ ensures access to the latest language features and AI ecosystem libraries.

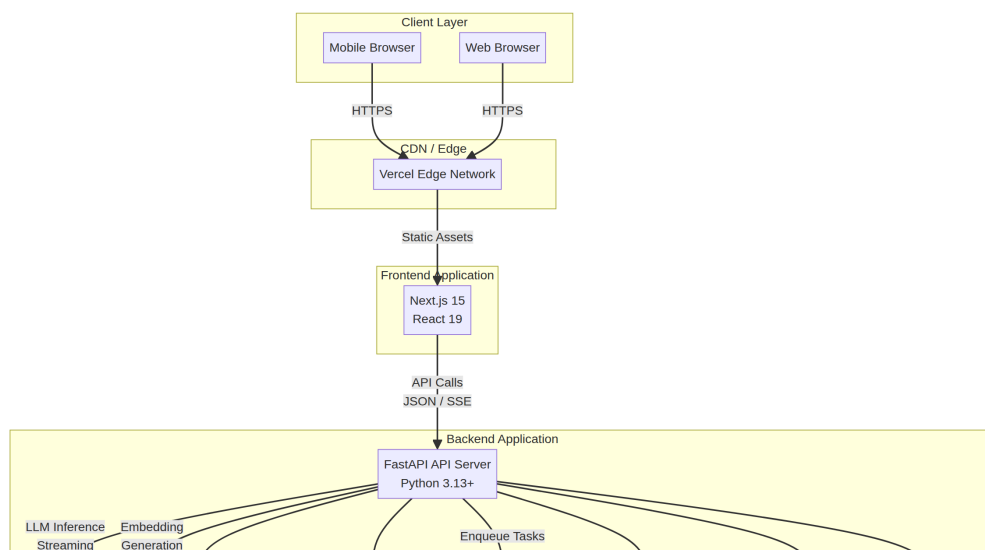
		Trade-offs: Python's Global Interpreter Lock (GIL) can be a bottleneck for CPU-bound tasks, though async execution mitigates this for I/O-bound AI operations.
Database	Neon PostgreSQL	Rationale: Neon provides a serverless, scalable, and branchable PostgreSQL database, aligning with cloud-native principles. It simplifies operational overhead and supports point-in-time recovery. Trade-offs: Serverless architectures can introduce cold start latencies for the first connection.
Vector Database	Qdrant	Rationale: Qdrant is a high-performance, Rust-based vector search engine that excels at scaling and complex filtering. It supports hybrid search and payload-based filtering, which is crucial for enterprise RBAC. Trade-offs: Requires separate infrastructure management compared to managed relational databases with vector extensions (e.g., pgvector).
Object Storage	Backblaze B2	Rationale: Backblaze B2 offers highly scalable, cost-effective S3-compatible object storage, ideal for storing large enterprise documents and media files. Trade-offs: Egress costs and bandwidth limits need to be monitored, though they are generally lower than primary cloud providers.
Embedding	sentence-transformers/paraphrase-	Rationale: This model provides a good balance

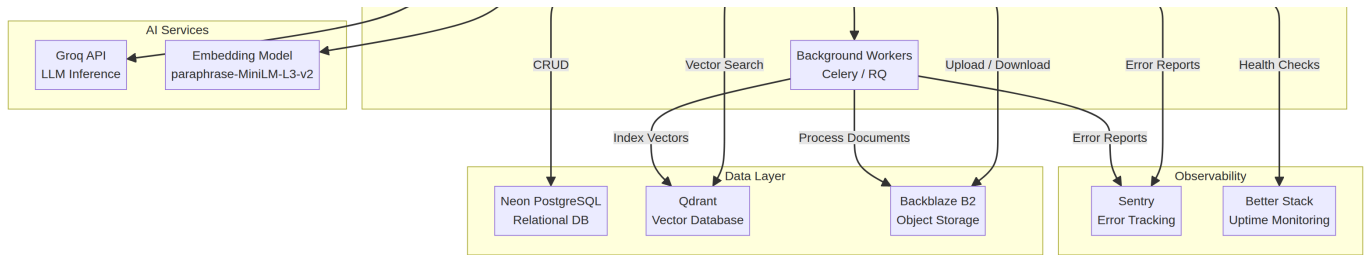
	MiniLM-L3-v2	between embedding quality, speed, and computational efficiency (384 dimensions), suitable for rapid semantic matching. Trade-offs: While efficient, it may lack the deep semantic understanding of larger models like <code>bge-large</code> or <code>OpenAI embeddings</code>, potentially reducing accuracy for highly specialized enterprise jargon.
LLM	Groq	Rationale: Groq provides ultra-fast inference speeds, which is critical for a responsive streaming chat experience. The architecture includes a provider abstraction to allow future switching to OpenAI, Anthropic, or Gemini. Trade-offs: Currently limited to specific open-source models, which may not match the absolute peak performance of proprietary models like GPT-4, though speed often compensates for this in real-time applications.
Authentication	JWT, Refresh Tokens, Google OAuth, GitHub OAuth, RBAC	Rationale: JWTs provide stateless, scalable session management. OAuth integrations streamline enterprise user onboarding. RBAC ensures granular access control. Trade-offs: JWTs must be carefully managed (short expiration, secure storage) to prevent token theft.
Deployment	Vercel (Frontend), Railway/Render (Backend)	Rationale: Vercel offers optimized edge deployment for Next.js. Railway and Render provide simple, developer-friendly managed backend hosting with good CI/CD integration.

		Trade-offs: Platform lock-in and potential limitations in custom infrastructure configurations compared to raw Kubernetes or AWS ECS.
CI/CD	GitHub Actions	Rationale: Deep integration with GitHub repositories, robust marketplace of actions, and excellent parallelization capabilities. Trade-offs: Cost scaling based on usage, though generally manageable for standard development workflows.
Monitoring	Sentry, Better Stack	Rationale: Sentry provides excellent application performance monitoring (APM) and error tracking. Better Stack offers reliable uptime monitoring and incident management. Trade-offs: Requires managing separate accounts and integrating multiple dashboards for a unified view.

8. High-Level Architecture

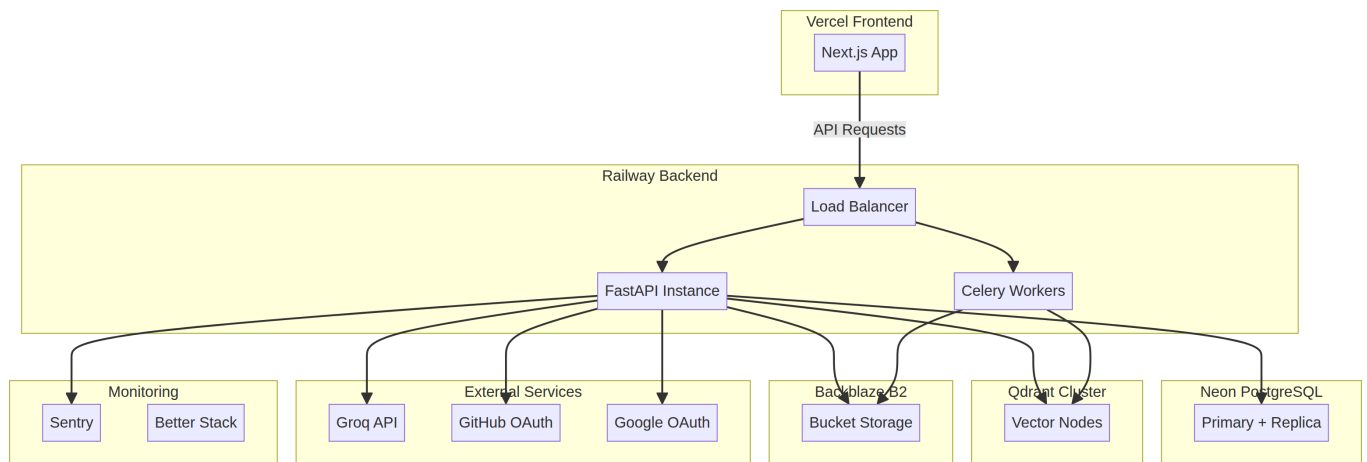
The high-level architecture illustrates the interaction between the client, backend services, and external integrations.





9. Deployment Architecture

The deployment architecture outlines how the system is hosted and managed in production.



10. Component Architecture

Frontend

The frontend is built as a modular Next.js application, utilizing React 19 for component rendering. TanStack Query handles server-state synchronization, while Zustand manages ephemeral UI state (e.g., chat streaming states, modals). Tailwind CSS and shadcn/ui provide a consistent, accessible design system.

Backend

The backend is structured as a FastAPI application. It exposes RESTful endpoints for CRUD operations and streaming endpoints (SSE/WebSockets) for the AI chat interface. It utilizes Pydantic for strict request/response validation and dependency injection for database and service clients.

Database

Neon PostgreSQL serves as the primary relational database, storing user profiles, organization metadata, document records, and access control lists. It employs row-level

security (RLS) where applicable to enforce multi-tenancy.

Vector Database

Qdrant manages the vector embeddings for documents. It stores high-dimensional vectors alongside payload metadata (e.g., document ID, tenant ID, chunk index) to enable efficient filtering during retrieval.

Storage

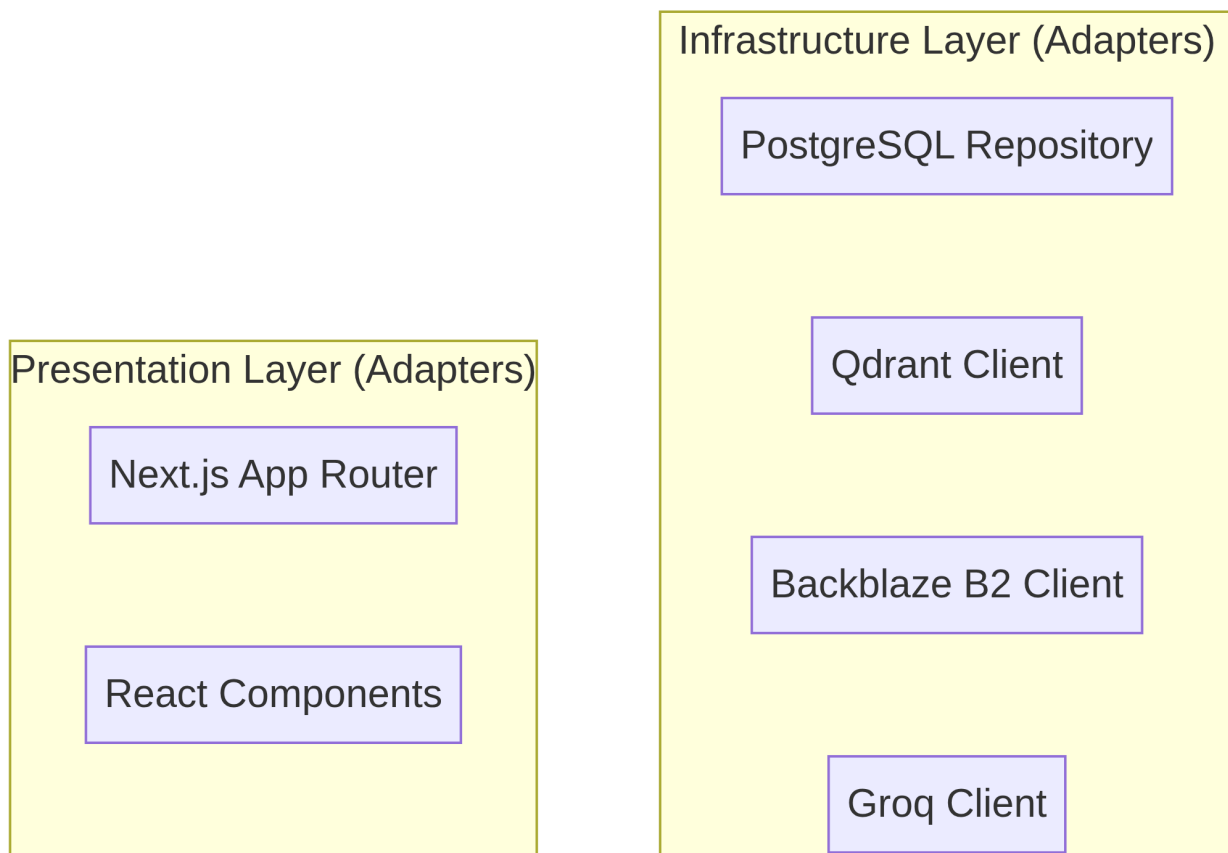
Backblaze B2 handles the physical storage of uploaded documents. Files are stored in tenant-isolated buckets or paths, with lifecycle policies configured for automatic archival or deletion.

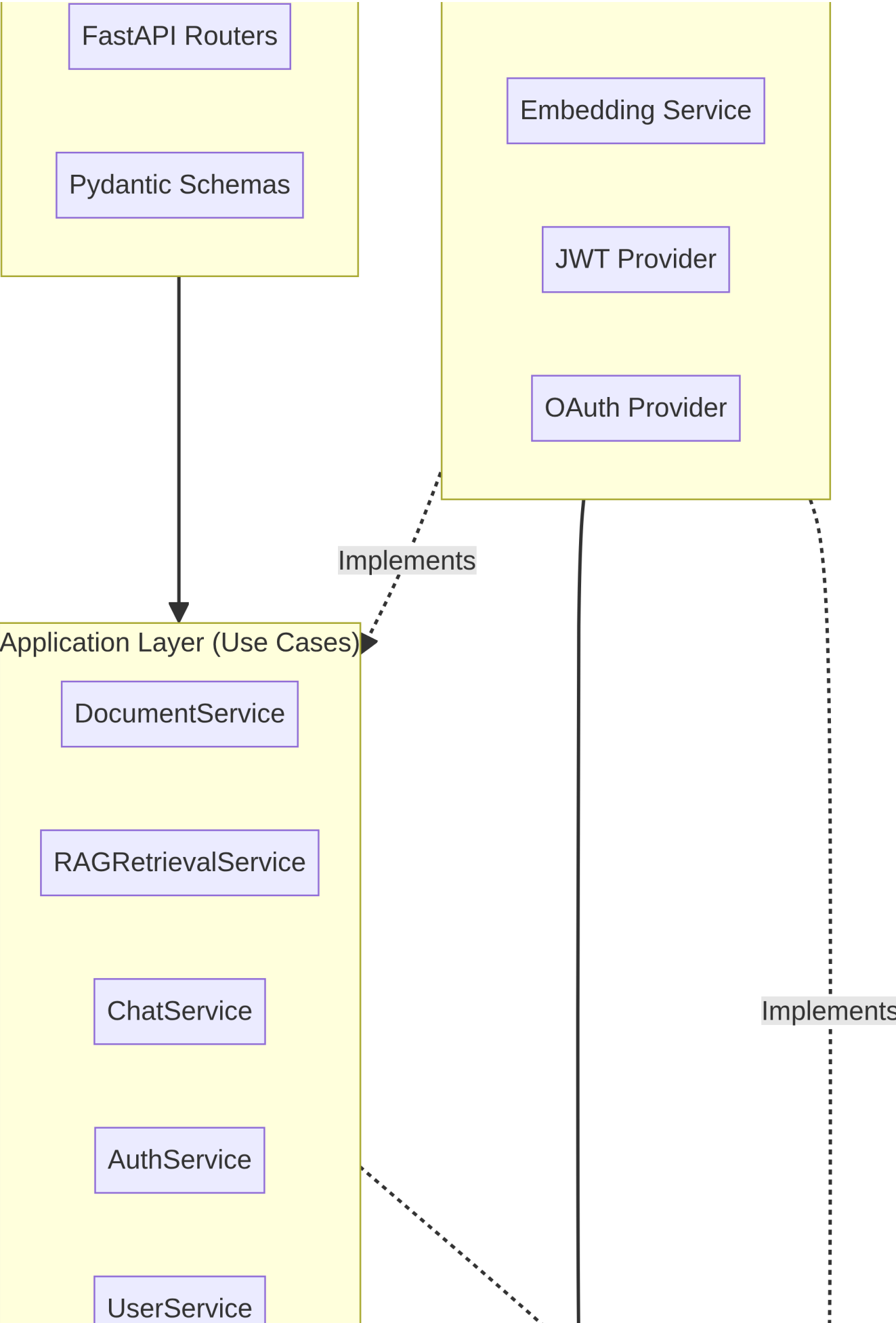
AI Services

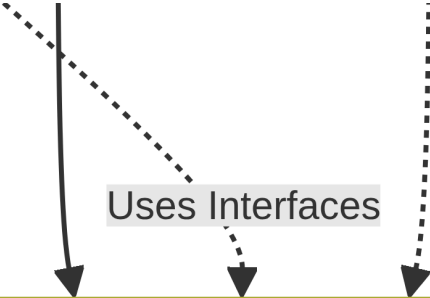
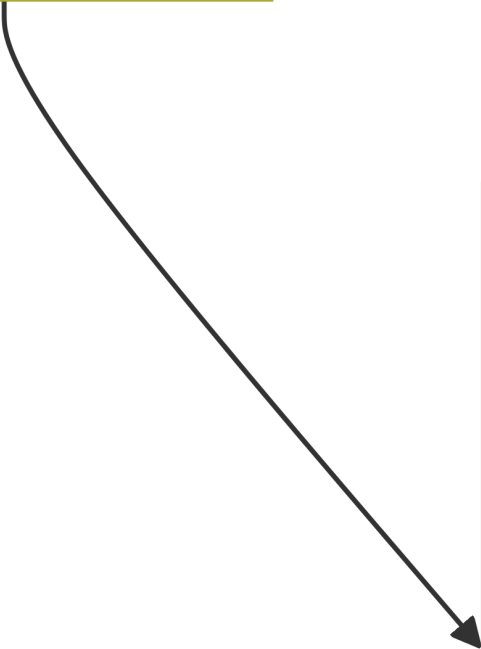
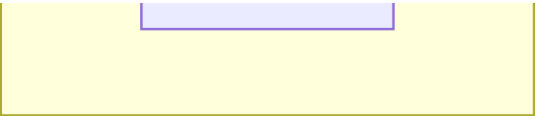
The AI services component abstracts the interaction with Groq for text generation and the local embedding model for vector generation. It includes retry logic, rate limiting, and context window management.

11. Clean Architecture

The application adheres to Clean Architecture principles, ensuring that the core business logic is independent of the UI, database, and external frameworks.







Domain Layer (Entities & Rules)

Document Entity

User Entity

Organization Entity

Chunk Entity

Conversation Entity

Repository Interfaces

Use Case Interfaces

- **Presentation (Adapters):** The Next.js frontend and FastAPI controllers handle HTTP requests, rendering, and user interactions.
- **Application (Use Cases):** Defines application-specific business rules. It orchestrates data flow between the domain and infrastructure layers.
- **Domain (Entities):** Contains the core business objects (e.g., `Document` , `User` , `Organization`) and their logic. This layer has no dependencies on outer layers.
- **Infrastructure (Adapters):** Implements technical concerns such as database repositories, vector store clients, and external API integrations (e.g., Groq, Backblaze B2).

Dependency Rule: Source code dependencies must only point inward. The Domain layer is at the center, depending on nothing. The Application layer depends on the Domain. The Infrastructure and Presentation layers depend on the Application and Domain layers.

12. Frontend Architecture

- **Folder Structure:** Organized by feature (e.g., `components/auth` , `components/chat` , `hooks` , `services`) to promote scalability and maintainability.
- **State Management:** TanStack Query for server data (documents, search results), Zustand for client state (chat history, UI toggles).
- **Routing:** Next.js App Router for file-based routing, protecting routes via middleware authentication checks.
- **Caching:** Aggressive caching for static assets and document metadata using TanStack Query's stale-while-revalidate strategy.
- **Authentication Flow:** Users log in via OAuth or credentials. The frontend stores the JWT in a secure, httpOnly cookie and uses it to authenticate API requests.
- **Streaming UI:** Utilizes the `fetch` API with `ReadableStream` to handle SSE from the backend, updating the chat interface token-by-token.
- **Accessibility:** Follows WAI-ARIA guidelines, utilizing `shadcn/ui` primitives and semantic HTML.
- **Error Handling:** Global error boundaries in React and centralized error handling in TanStack Query to display user-friendly notifications.

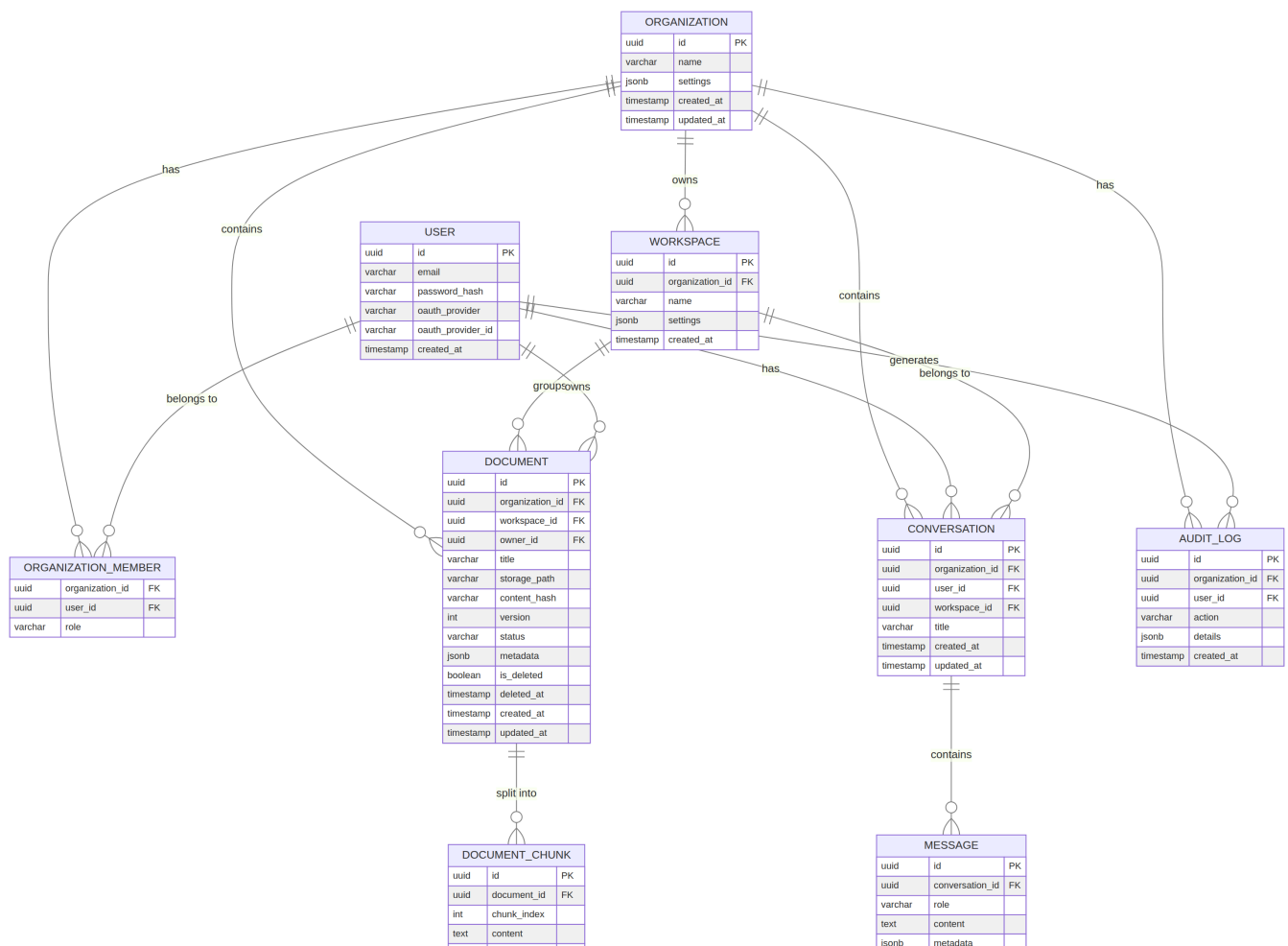
13. Backend Architecture

- **Folder Structure:** Modular structure (`src/api` , `src/core` , `src/infrastructure` , `src/domain`).

- **Dependency Injection:** Uses FastAPI's `Depends` for injecting database sessions, repository instances, and service classes.
- **Services:** Encapsulate complex business logic (e.g., `DocumentProcessingService`, `RAGRetrievalService`).
- **Repositories:** Abstract database interactions, returning domain entities.
- **Controllers (Routers):** FastAPI routers handle HTTP routing and request parsing, delegating logic to services.
- **Middlewares:** CORS, authentication (JWT verification), rate limiting, and logging.
- **Workers:** Celery or RQ workers process asynchronous tasks like document parsing and embedding generation.
- **Validation:** Pydantic models ensure strict type checking and validation of all incoming requests.
- **Configuration:** Managed via environment variables using Pydantic's `BaseSettings`.

14. Database Design

The PostgreSQL schema supports multi-tenancy, soft deletes, and audit trails.



id	SUMMARY_USERS	
jsonb	metadata	

jsonb	citations	
timestamp	created_at	

SQL

-- Organizations and Users

```
CREATE TABLE organizations (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(255) NOT NULL,
  settings JSONB DEFAULT '{}',
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

```
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email VARCHAR(255) UNIQUE NOT NULL,
  password_hash VARCHAR(255),
  oauth_provider VARCHAR(50),
  oauth_provider_id VARCHAR(255),
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

```
CREATE TABLE organization_members (
  organization_id UUID REFERENCES organizations(id),
  user_id UUID REFERENCES users(id),
  role VARCHAR(50) NOT NULL, -- e.g., 'admin', 'member', 'viewer'
  PRIMARY KEY (organization_id, user_id)
);
```

-- Documents

```
CREATE TABLE documents (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  organization_id UUID REFERENCES organizations(id),
  owner_id UUID REFERENCES users(id),
  title VARCHAR(255) NOT NULL,
  storage_path VARCHAR(1024) NOT NULL,
  content_hash VARCHAR(64) NOT NULL,
  version INT DEFAULT 1,
  status VARCHAR(50) DEFAULT 'pending', -- 'pending', 'processing', 'complete'
  metadata JSONB DEFAULT '{}',
  is_deleted BOOLEAN DEFAULT FALSE,
  deleted_at TIMESTAMP WITH TIME ZONE,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

-- Indexes

```
CREATE INDEX idx_documents_org ON documents(organization_id);
CREATE INDEX idx_documents_status ON documents(status);
```

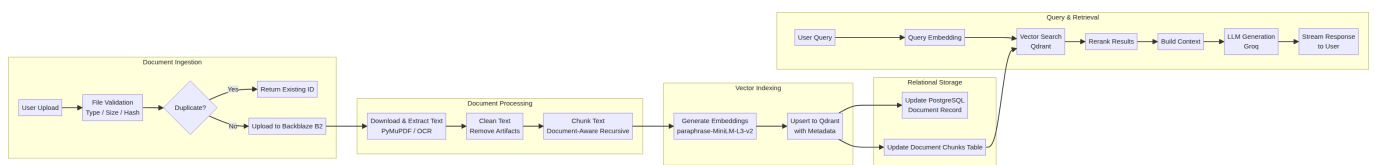
15. Vector Database Design

- **Collections:** Organized by `organization_id` to ensure strict multi-tenancy.
- **Payload Schema:** Stores `document_id` , `chunk_index` , `document_version` , and `access_control_metadata` .
- **Metadata Filtering:** Qdrant payloads are indexed to allow efficient filtering during vector search (e.g., finding vectors belonging to specific documents or users).
- **Embedding Dimensions:** 384 dimensions (matching `paraphrase-MiniLM-L3-v2`).
- **Similarity Metric:** Cosine similarity, suitable for semantic text matching.

16. Object Storage Design

- **Backblaze B2 Layout:** `s3://<bucket-name>/<organization_id>/<document_id>/<version>/file.ext` .
- **Versioning:** Enabled at the bucket level to maintain historical copies of documents.
- **Lifecycle:** Automated rules to transition old versions to Cold Storage or delete them after a retention period.
- **Metadata:** Object metadata includes `organization_id` , `user_id` , `file_type` , and `content_hash` .

17. Document Processing Pipeline



1. **Upload:** User uploads a file via the frontend; frontend sends it to Backblaze B2 (via pre-signed URL) and registers it in the backend database.
2. **Validation:** Backend verifies file type, size, and hash.
3. **Hashing:** Generates SHA-256 hash to detect duplicates and ensure integrity.
4. **Duplicate Detection:** Checks if the hash already exists in the database.
5. **OCR (Optional):** If the document is an image/scanned PDF, runs Tesseract or AWS Textract.
6. **Extraction:** Parses text using libraries like `PyMuPDF` or `pdfplumber` .

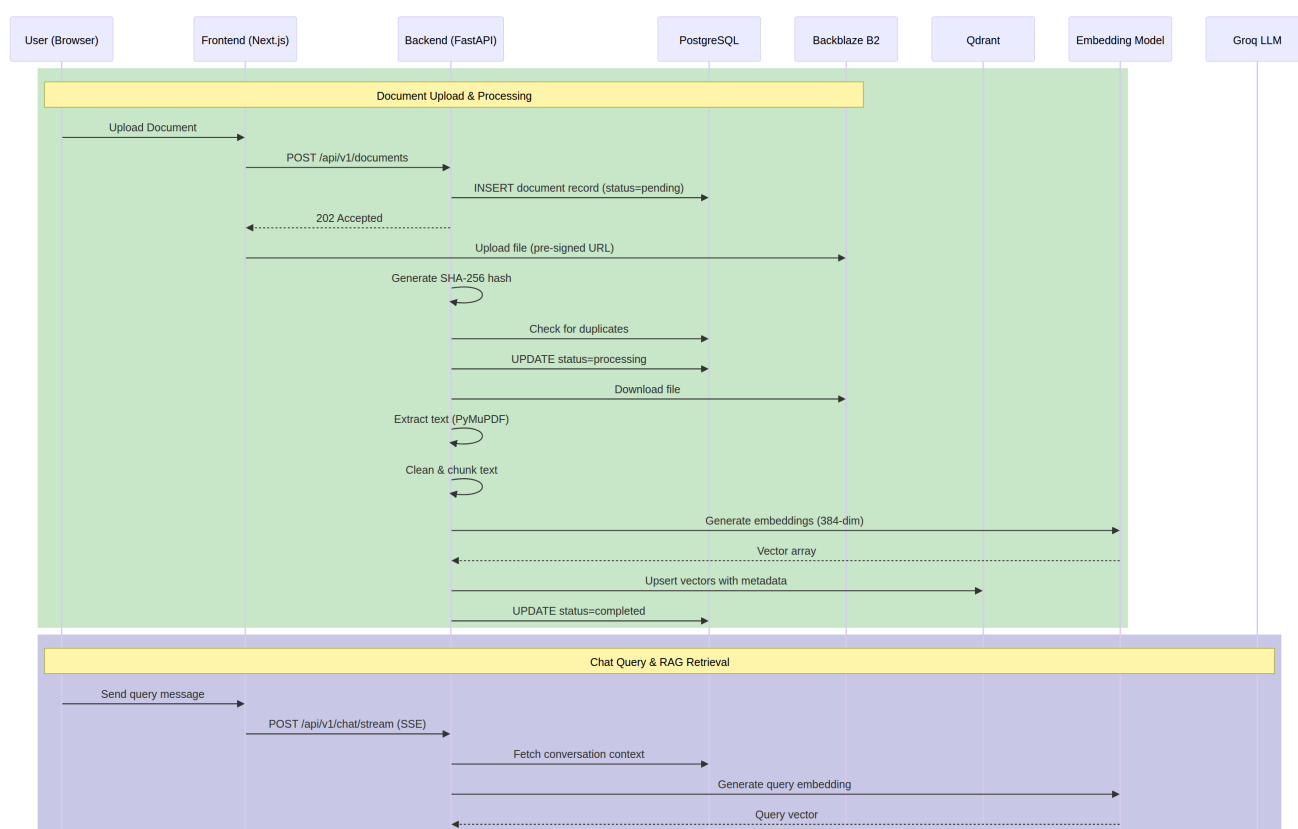
7. **Cleaning:** Removes formatting artifacts, headers, footers, and redundant whitespace.
8. **Chunking:** Splits the cleaned text into semantic chunks.
9. **Embedding:** Generates 384-dimensional vectors for each chunk.
10. **Indexing:** Upserts the vectors and payload metadata into Qdrant.

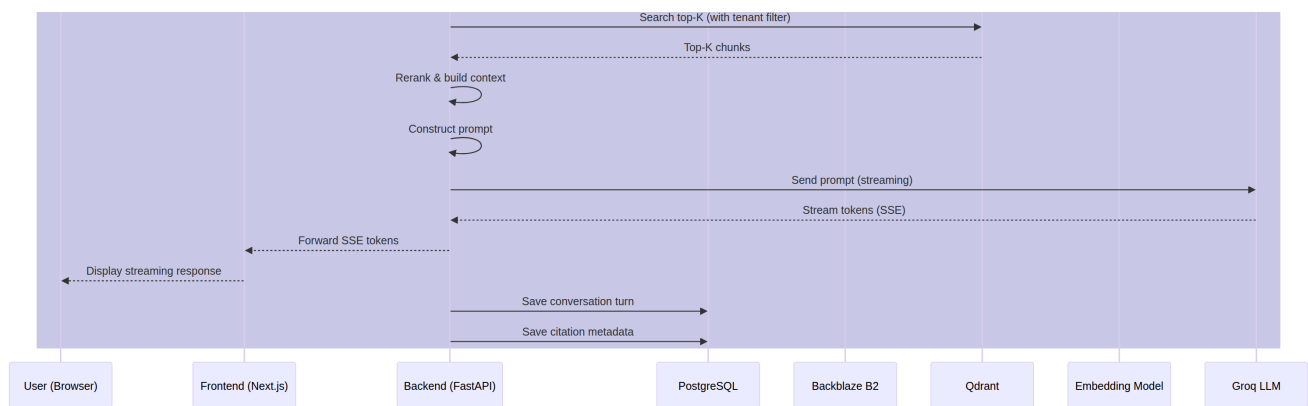
18. Chunking Strategy

The platform employs **Document-Aware Recursive Chunking** to maximize retrieval accuracy and context preservation ¹.

- **Recursive Chunking:** The algorithm attempts to split text using hierarchical separators (e.g., `\n\n`, `\n`, `.`). It prioritizes larger chunks that maintain semantic boundaries.
- **Document-Aware Chunking:** Before recursive splitting, the text is analyzed to identify logical sections (e.g., Markdown headers `#`, `##`). Chunks are aligned to these sections whenever possible.
- **Chunk Overlap:** A 10-20% overlap is maintained between adjacent chunks to ensure context is not lost at the boundaries.
- **Trade-offs:** Document-aware chunking requires robust PDF/Markdown parsers. It may fail on unstructured text, in which case it falls back to standard recursive chunking.

19. Retrieval Pipeline



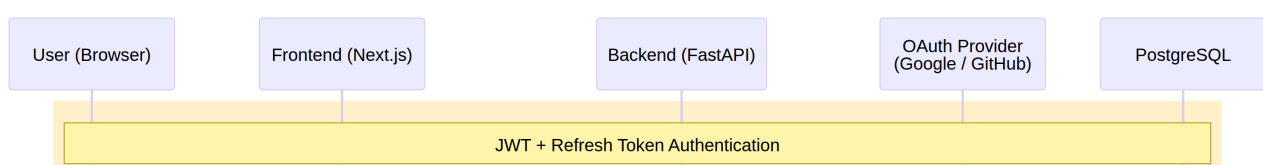


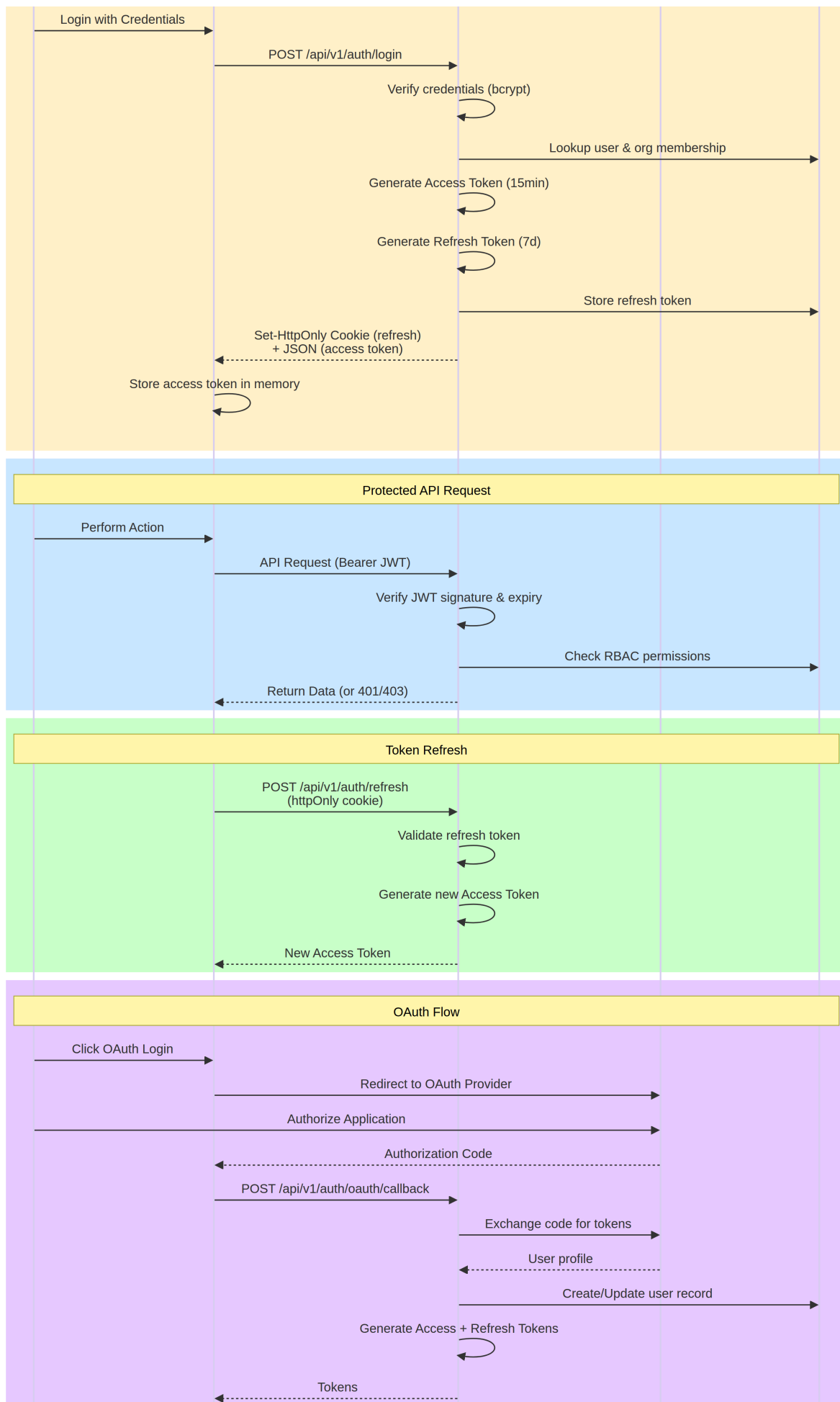
1. **Query Preprocessing:** Normalizes the user's query (lowercase, remove punctuation).
2. **Embedding Generation:** Generates a 384-dimensional vector for the query.
3. **Vector Search:** Queries Qdrant for the top-K nearest neighbors, applying metadata filters (e.g., `organization_id`).
4. **Reranking:** (Optional) Re-scores the top-K results using a cross-encoder model for higher precision.
5. **Context Building:** Concatenates the top-ranked chunks into a single context string.
6. **Prompt Construction:** Assembles the final prompt, including system instructions, the user query, and the retrieved context.
7. **LLM Generation:** Sends the prompt to Groq, streaming the response back to the user.
8. **Citation Generation:** Maps the generated text back to the source chunks to provide citations.

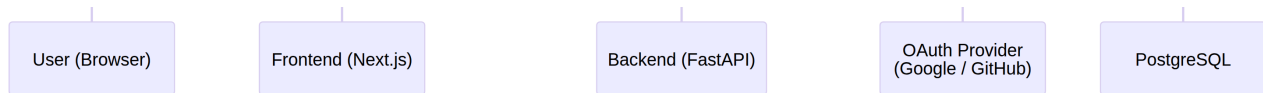
20. Conversation Management

- **Sessions:** Each chat is a distinct session stored in PostgreSQL, linked to the user and organization.
- **History:** Stores user messages and AI responses.
- **Context Window:** Manages the LLM's context window by summarizing older messages or truncating them.
- **Memory Strategy:** Implements a sliding window approach, retaining the most recent N messages and injecting a summary of earlier messages.

21. Authentication







- **JWT:** Short-lived access tokens (e.g., 15 minutes) issued upon successful login.
- **Refresh Tokens:** Long-lived tokens (e.g., 7 days) stored in httpOnly cookies, used to obtain new access tokens.
- **OAuth:** Integration with Google and GitHub OAuth for seamless enterprise onboarding.
- **RBAC:** Roles (Admin, Editor, Viewer) determine the user's capabilities within the system.
- **Permissions:** Fine-grained permissions (e.g., `document:create` , `chat:execute`) are enforced at the API level.
- **Security:** Passwords are hashed using bcrypt. JWT secrets are stored securely.

22. Authorization

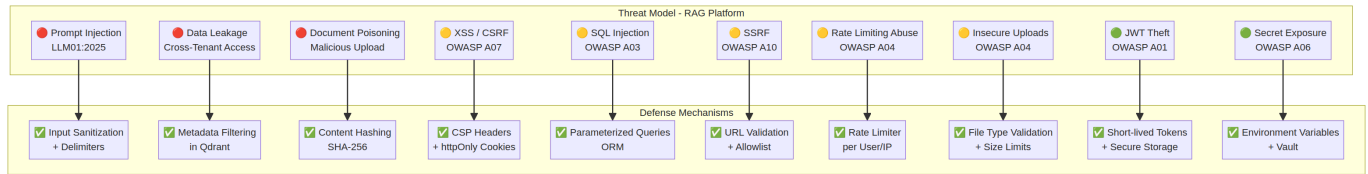
- **Organization:** Users belong to one or more organizations. Data is strictly scoped to the user's active organization.
- **Workspace:** Documents and chat sessions are grouped into workspaces.
- **Document:** Access control lists (ACLs) manage who can view or edit specific documents.
- **Admin:** Organization admins can manage members, billing, and global settings.
- **Ownership:** The creator of a document owns it and can transfer ownership.

23. API Design

- **REST Endpoints:** Standard RESTful conventions (`GET /documents` , `POST /documents` , etc.).
- **Request Models:** Strict Pydantic models defining required fields, types, and constraints.
- **Response Models:** Consistent JSON structures including `data` , `meta` (pagination), and `error` objects.
- **Pagination:** Cursor-based pagination for infinite scrolling in chat and list views.
- **Filtering & Sorting:** Query parameters for filtering lists (e.g., `?status=completed&sort=created_at:desc`).
- **Versioning:** API versioning via URL paths (e.g., `/api/v1/`).
- **Streaming Endpoints:** `/api/v1/chat/stream` uses Server-Sent Events (SSE) for real-time token streaming.

- **Error Responses:** Standardized HTTP status codes and detailed error messages (e.g., 404 Not Found , 403 Forbidden , 400 Bad Request).

24. Security Architecture



- **OWASP Top 10:** Mitigations for Injection, Broken Access Control, and Security Misconfigurations.
- **Prompt Injection:** Input validation and sanitization on user queries. Use of delimiters in prompt construction to separate context from user input 2 .
- **Data Leakage:** Strict RBAC and metadata filtering in Qdrant to prevent cross-tenant data leakage.
- **Secrets Management:** Environment variables managed via Railway/Render secrets UI; no hardcoded secrets in code.
- **Rate Limiting:** API rate limits applied per user/IP to prevent abuse.
- **XSS & CSRF:** Content Security Policy (CSP) headers, httpOnly cookies for refresh tokens, and state parameter validation for OAuth.
- **SQL Injection:** Parameterized queries via ORMs (e.g., SQLAlchemy).
- **SSRF:** Validation of user-provided URLs before fetching external content.
- **Secure Uploads:** File type validation, size limits, and storage in isolated buckets.
- **Audit Logs:** Comprehensive logging of authentication events, document uploads, and AI interactions.

25. Performance Optimization

- **Caching:** Redis (or native FastAPI caching) for frequently accessed document metadata and vector search results.
- **Connection Pooling:** Database connection pooling to handle high concurrency.
- **Compression:** Gzip/Brotli compression enabled for API responses.
- **Lazy Loading:** Frontend components and images loaded on demand.
- **Streaming:** FastAPI `StreamingResponse` for AI chat to reduce Time to First Token (TTFT).
- **Indexes:** Proper indexing on PostgreSQL (B-tree, GIN for JSONB) and Qdrant (HNSW).

- **Async Tasks:** Offloading heavy processing (PDF parsing, embedding) to background workers.

26. Scalability

- **Horizontal Scaling:** Frontend and backend services scale horizontally behind load balancers.
- **Database Scaling:** Neon PostgreSQL supports read replicas and compute scaling.
- **Qdrant Scaling:** Qdrant clusters can be scaled horizontally by adding more nodes.
- **Storage Scaling:** Backblaze B2 scales infinitely for object storage.
- **Background Jobs:** Task queues (e.g., Celery) scale independently of the main application.

27. Background Processing

- **Document Processing:** Async workers handle PDF parsing, OCR, and text extraction.
- **Embedding Generation:** Workers generate vectors and update Qdrant.
- **OCR:** Triggered automatically for scanned documents.
- **Notifications:** Sending email or in-app notifications upon task completion.
- **Cleanup:** Scheduled jobs to archive old documents and clear temporary files.

28. Deployment Strategy

- **Vercel:** Automated deployments on Git push for the frontend, leveraging edge functions.
- **Railway / Render:** Continuous deployment pipelines for the backend.
- **Environment Variables:** Managed securely via platform UIs.
- **Secrets:** Rotated regularly; injected at runtime.
- **Rollback:** Automated rollback to the previous version on deployment failure.
- **Blue-Green Deployment:** Supported by Railway/Render to minimize downtime during updates.

29. Observability

- **Logging:** Structured JSON logging (e.g., using `structlog`) capturing request IDs, user IDs, and error traces.

- **Metrics:** Application metrics (request duration, error rates) exported to monitoring tools.
- **Tracing:** Distributed tracing (e.g., OpenTelemetry) to track requests across services.
- **Alerts:** Configured alerts in Sentry and Better Stack for high error rates or downtime.
- **Health Checks:** `/health` endpoints for liveness and readiness probes.
- **Dashboards:** Grafana or native platform dashboards to visualize system performance.

30. CI/CD

- **GitHub Actions:** Automated workflows for testing, linting, and deploying.
- **Testing:** Unit, integration, and end-to-end tests executed on pull requests.
- **Linting:** ESLint for frontend, Ruff/Black for backend.
- **Formatting:** Prettier for frontend, Black for backend.
- **Security Scanning:** Snyk or Dependabot to detect vulnerabilities in dependencies.

31. Testing Strategy

- **Unit:** Testing individual functions and classes in isolation (e.g., chunking logic, prompt construction).
- **Integration:** Testing interactions between services (e.g., FastAPI to PostgreSQL).
- **API:** Testing HTTP endpoints for correct responses and error handling.
- **Load:** Simulating high traffic to ensure the system scales effectively.
- **Security:** Automated scans for OWASP vulnerabilities and penetration testing.
- **End-to-End:** Playwright or Cypress tests simulating user interactions across the full stack.

32. Coding Standards

- **Folder Structure:** Consistent across frontend and backend projects.
- **Naming:** CamelCase for variables/functions, PascalCase for classes/components, snake_case for database fields.
- **Error Handling:** Custom exception classes, centralized error handlers, and no silent failures.
- **Documentation:** Comprehensive docstrings and a centralized architecture document.
- **Logging:** Standardized log levels (INFO, WARN, ERROR) and structured formats.

- **Dependency Rules:** Strict adherence to Clean Architecture dependency rules.

33. Project Structure

Plain Text

```
enterprise-rag-platform/
├── frontend/                # Next.js 15 Application
│   ├── src/
│   │   ├── app/            # App Router
│   │   ├── components/    # Reusable UI Components
│   │   ├── hooks/         # Custom React Hooks
│   │   ├── lib/           # Utilities and API Clients
│   │   ├── store/         # Zustand State Management
│   │   └── types/         # TypeScript Interfaces
│   ├── public/            # Static Assets
│   └── package.json
├── backend/                # FastAPI Application
│   ├── src/
│   │   ├── api/           # Routers and Endpoints
│   │   ├── core/          # Configuration and Security
│   │   ├── domain/        # Entities and Interfaces
│   │   ├── infrastructure/ # DB, Vector DB, Storage Clients
│   │   ├── services/      # Business Logic
│   │   └── workers/       # Celery Tasks
│   ├── tests/             # Pytest Suite
│   └── requirements.txt
└── README.md
```

34. Development Roadmap

- **Phase 1 (MVP):** Core document upload, basic vector search, and simple chat interface.
- **Phase 2 (Enhancements):** Advanced chunking strategies, RBAC, and multi-tenancy support.
- **Phase 3 (Optimization):** Background processing pipelines, streaming responses, and performance tuning.
- **Phase 4 (Enterprise):** Audit logs, advanced security features, and analytics dashboards.

35. Risk Analysis

- **Technical:** LLM hallucination leading to incorrect citations. Mitigated by strict prompt engineering and reranking.

- **Business:** High API costs from Groq or Backblaze B2. Mitigated by caching and quota management.
- **Security:** Prompt injection attacks. Mitigated by input sanitization and context window delimiters.
- **Operational:** Downtime during backend deployments. Mitigated by blue-green deployments.
- **AI:** Degradation in embedding model performance. Mitigated by continuous evaluation and fallback strategies.
- **Scalability:** Qdrant bottleneck under heavy load. Mitigated by horizontal scaling and HNSW tuning.

36. Future Enhancements

- **Enterprise Roadmap:** Single Sign-On (SSO) via SAML/OIDC.
- **Multi-region:** Deploying backend services in multiple geographical regions for lower latency.
- **Multi-cloud:** Abstracting infrastructure to support AWS, Azure, and GCP.
- **AI Agents:** Implementing autonomous agents capable of executing complex multi-step tasks.
- **Hybrid Search:** Combining vector search with keyword search (e.g., BM25) for improved recall.
- **Advanced RAG:** Implementing GraphRAG or hierarchical retrieval for complex document structures.

37. References

- [1] Weaviate. "Chunking Strategies to Improve LLM RAG Pipeline Performance."
- [2] OWASP. "Retrieval-Augmented Generation (RAG) Security Cheat Sheet."